الدخول والجلسات في زمن الاختلاسات

Lecture 3 — Authentication and Session Management

> **Authentication (AuthN)** : answers "**who are you?**" — proving identity (e.g. username + password).

> **Authorization (AuthZ)** : answers "**what are you allowed to do?**" — which resources and actions the known identity may use.

Authentication	Authorization
Who are you?	What may you do?
Fails → 401 (identity unknown)	Fails → **403** (identity known, action denied)
Comes first	Comes after identity is verified

> **Authentication factor** : a category of evidence used to prove identity.

Factor	Examples	Weakness if used alone
Something you know	Password, PIN, security question	Guessed, phished, or reused from another breach
Something you have	Phone (OTP app / SMS), hardware key (YubiKey)	Lost, stolen, or SMS intercepted (SIM swap)
Something you are	Fingerprint, face recognition	Can't be "reset" if the biometric data is compromised

> **MFA (Multi-Factor Authentication)** : combining factors from at least **two different categories**.

Q/ Why is MFA stronger than one factor ?

- It forces the attacker to defeat **two unrelated categories** at once; a leaked password alone is no longer enough.

Two passwords = still one factor (both "something you know").

Password storage

Q/ How must passwords NOT be stored ?

1. In **plaintext**.
2. With **reversible encryption** (they must never be recoverable).

> **Hashing** : a one-way function. Login = re-hash the entered password and **compare hashes** — never decrypt.

> **Salt** : a unique random value per user added before hashing, so identical passwords give different hashes — defeats **rainbow tables**.

> **Slow hash** : deliberately expensive algorithm so guessing millions of passwords per second becomes costly.

Q/ Rank the acceptable password hashing algorithms (OWASP).

1. **Argon2id** — first choice.
2. **scrypt** — fallback.
3. **bcrypt** — legacy systems.
4. **PBKDF2** — where FIPS-140 compliance is required.

Plain **MD5 or SHA-256 are NOT acceptable** for passwords: they're designed to be **fast**, exactly the wrong property here.

Sessions

> **Session ID** : a long, random, unguessable value issued after login via **Set-Cookie**; the browser resends it with every request and the server looks it up in its **session store**.

Q/ What two operational rules matter for sessions ?

1. **Idle timeout**: sessions expire after inactivity.
2. **Regenerate the session ID at login**: always issue a fresh ID, replacing whatever the browser held before.

> **Session Fixation** : the attacker gets the victim to authenticate using a session ID **the attacker already knows**; if the server keeps the same ID after login, the attacker is now logged in as the victim. The attacker never touches the victim's credentials.

Fix: **every privilege change** (login, elevating to admin) issues a **brand-new session ID**.

Attack	How it works	Primary mitigation
Brute force	Automated tool tries many passwords against one account	Account lockout / rate limiting, CAPTCHA, MFA
Credential stuffing	Replays username/password pairs leaked from **other** breaches	MFA, check new passwords against breach lists, per-IP rate limiting
Session hijacking	Steals a valid session ID via XSS, sniffing plain HTTP, or a leaked log	**HttpOnly + Secure** cookies, HTTPS everywhere
Session fixation	Victim logs in with an attacker-known session ID	**Regenerate** the session ID at login

Brute force	Credential stuffing
Guesses passwords	Reuses real leaked pairs
One account, many guesses	Many accounts, one leaked pair each
Lockout helps	MFA and breach-list checks help

Modern authentication

> **Token-based authentication (JWT)** : a self-contained token of three **base64url** parts — `header.payload.signature`.

Q/ What does each JWT part contain ?

1. **Header**: the signing algorithm.
2. **Payload**: claims (who the user is, when the token expires).
3. **Signature**: created with a server-held secret or private key, so the server can verify the token wasn't tampered with **without storing it**.

Session cookie	JWT
Server keeps a lookup table of sessions	Stateless: nothing stored server-side
Revoke by deleting the session entry	A stolen token stays **valid until it expires** (no entry to revoke)

> **OAuth 2.0 / SSO (Single Sign-On)** : the user authenticates **once** with a trusted identity provider (Google, Microsoft, university account) and reuses that identity across many applications — the app avoids storing passwords at all.

Q/ What is the common theme behind session fixation ?

- Trusting something that should have been **regenerated** — the same lesson as Weeks 1–2: don't trust values the client (or attacker) controls.

Summary of Web Applications Security L3: Authentication & Session Management. Built only from the lecture file; check the original slides for anything you're unsure about.